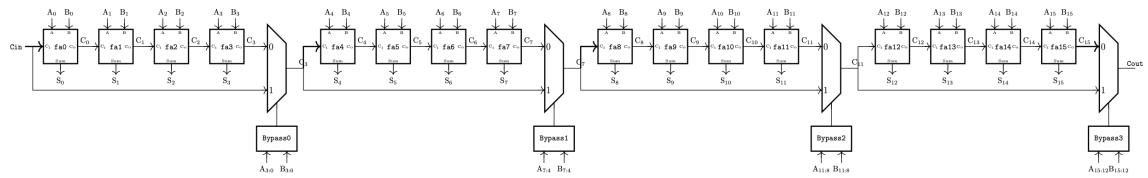


EE5311 Digital Circuits and Simulation
Tutorial 11: False path in carry bypass adder
Venkata Subhash M - NS25Z205

Assignment Questions

EE5311 Digital IC Design: Assignment 11
False path in carry bypass adder

1. Implement the below carry-bypass adder in Verilog:



2. Add a clock input and register the inputs/outputs of the above bypass adder.
3. Synthesize the adder using Yosys with the Sky130 PDK.
4. Perform STA, report the critical path and its slack.
5. Is the reported critical path right? If it is not, get STA to report the right critical path and its slack?

Figure 1: Tutorial 11 Assignment Questions

Section 1 & 2: Carry Bypass verilog implemntation

This section covers the registered version of the 16-bit Ripple Carry Bypass Adder implementation with input and output registers for synchronous operation.

```
module cb (
    input          clk ,
    input  [15:0]  A,
    input  [15:0]  B,
    input          Cin ,
    output reg [15:0] Sum,
    output reg          Cout
);
    reg [15:0] A_r, B_r;
    reg          Cin_r;

    always @(posedge clk) begin
        A_r  <= A;
        B_r  <= B;
        Cin_r <= Cin;
    end

    wire [15:0] Sum_w;
    wire          Cout_w;

    wire [3:0] P0 = A_r[3:0] ^ B_r[3:0];
    wire [3:0] G0 = A_r[3:0] & B_r[3:0];
    wire          c1  = G0[0] | (P0[0] & Cin_r);
    wire          c2  = G0[1] | (P0[1] & c1);
    wire          c3  = G0[2] | (P0[2] & c2);
    wire          c4_ripple = G0[3] | (P0[3] & c3);
    wire          Pblock0   = &P0; // Bypass0 in the diagram
    wire          c4        = Pblock0 ? Cin_r : c4_ripple;
    assign Sum_w[3:0] = P0 ^ {c3, c2, c1, Cin_r};

    wire [3:0] P1 = A_r[7:4] ^ B_r[7:4];
    wire [3:0] G1 = A_r[7:4] & B_r[7:4];
    wire          c5  = G1[0] | (P1[0] & c4);
    wire          c6  = G1[1] | (P1[1] & c5);
    wire          c7  = G1[2] | (P1[2] & c6);
    wire          c8_ripple = G1[3] | (P1[3] & c7);
    wire          Pblock1   = &P1; // Bypass1
    wire          c8        = Pblock1 ? c4 : c8_ripple;
    assign Sum_w[7:4] = P1 ^ {c7, c6, c5, c4};

    wire [3:0] P2 = A_r[11:8] ^ B_r[11:8];
    wire [3:0] G2 = A_r[11:8] & B_r[11:8];
    wire          c9  = G2[0] | (P2[0] & c8);
    wire          c10 = G2[1] | (P2[1] & c9);
```

```

wire      c11  = G2[2] | (P2[2] & c10);
wire      c12_ripple = G2[3] | (P2[3] & c11);
wire      Pblock2    = &P2;                                // Bypass2
wire      c12        = Pblock2 ? c8 : c12_ripple;
assign    Sum_w[11:8] = P2 ^ {c11, c10, c9, c8};

wire [3:0] P3 = A_r[15:12] ^ B_r[15:12];
wire [3:0] G3 = A_r[15:12] & B_r[15:12];
wire      c13  = G3[0] | (P3[0] & c12);
wire      c14  = G3[1] | (P3[1] & c13);
wire      c15  = G3[2] | (P3[2] & c14);
wire      c16_ripple = G3[3] | (P3[3] & c15);
wire      Pblock3    = &P3;                                // Bypass3
wire      c16        = Pblock3 ? c12 : c16_ripple;
assign    Sum_w[15:12] = P3 ^ {c15, c14, c13, c12};
assign    Cout_w = c16;

always @(posedge clk) begin
    Sum <= Sum_w;

```

Section 3: Synthesis using yosys

The following image shows invoking YOSYS, reading the tt lib file and the Verilog file:

```
yosys> read_liberty -lib /cad/share/pdk/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd_tt_025C_1v80.lib
1. Executing Liberty frontend: /cad/share/pdk/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd_tt_025C_1v80.lib
Imported 428 cell types from liberty file.

yosys> read_verilog cb.v
2. Executing Verilog-2005 frontend: cb.v
Parsing Verilog input from `cb.v' to AST representation.
Generating RTLIL representation for module `cb'.
Successfully finished Verilog frontend.
```

The following image shows the synthesis command being run on the top module and successful synthesis

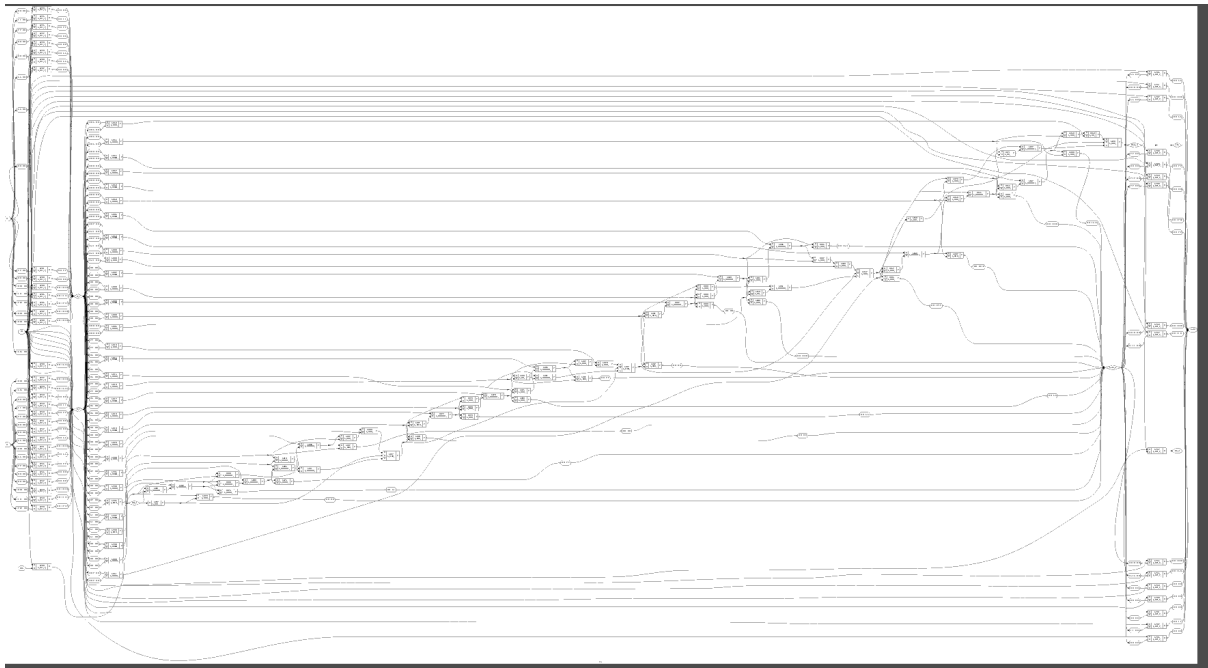
```
3.25. Printing statistics.

=== cb ===

Number of wires:          93
Number of wire bits:      183
Number of public wires:   12
Number of public wire bits: 102
Number of ports:          6
Number of port bits:      51
Number of memories:       0
Number of memory bits:    0
Number of processes:      0
Number of cells:          148
  $_ANDNOT_                20
  $_AND_                   4
  $_DFF_P_                 50
  $_MUX_                   4
  $_NAND_                  12
  $_NOR_                   19
  $_NOT_                   2
  $_ORNOT_                 1
  $_OR_                    4
  $_XNOR_                  16
  $_XOR_                   16

3.26. Executing CHECK pass (checking for obvious problems).
Checking module cb...
Found and reported 0 problems.
```

Synthesized Design of 16-bit Carr Bypass adder



Section 4: STA, critical path report and slack

The following image shows successfully reading the lib file, synthesized gate level netlist, setting the top module and creating the clock:

```
This is free software, and you are free to change and redistribute it
under certain conditions; type 'show copying' for details.
This program comes with ABSOLUTELY NO WARRANTY; for details type 'show warranty'.
% read_liberty /cad/share/pdk/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.lib
1
% read_verilog cb_synth.v
1
% link_design cb
1
% create_clock -name clk -period 3 {clk}
%
```

The following image shows the execution of report_checks command and the obtained critical path:

```
% create_clock -name clk -period 3 {clk}
% report_checks -path_delay max
Startpoint: _227_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _226_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

  Delay    Time    Description
-----
  0.00     0.00    clock clk (rise edge)
  0.00     0.00    clock network delay (ideal)
  0.00     0.00    ^ _227_/CLK (sky130_fd_sc_hd_dfxt_1)
  0.29     0.29    v _227_/Q (sky130_fd_sc_hd_dfxt_1)
  0.38     0.67    v _163_/X (sky130_fd_sc_hd_maj3_1)
  0.39     1.07    v _165_/X (sky130_fd_sc_hd_maj3_1)
  0.39     1.46    v _167_/X (sky130_fd_sc_hd_maj3_1)
  0.39     1.85    v _169_/X (sky130_fd_sc_hd_maj3_1)
  0.39     2.24    v _171_/X (sky130_fd_sc_hd_maj3_1)
  0.39     2.63    v _173_/X (sky130_fd_sc_hd_maj3_1)
  0.39     3.03    v _175_/X (sky130_fd_sc_hd_maj3_1)
  0.39     3.42    v _177_/X (sky130_fd_sc_hd_maj3_1)
  0.39     3.81    v _179_/X (sky130_fd_sc_hd_maj3_1)
  0.39     4.20    v _181_/X (sky130_fd_sc_hd_maj3_1)
  0.39     4.60    v _183_/X (sky130_fd_sc_hd_maj3_1)
  0.39     4.99    v _185_/X (sky130_fd_sc_hd_maj3_1)
  0.39     5.38    v _187_/X (sky130_fd_sc_hd_maj3_1)
  0.39     5.77    v _189_/X (sky130_fd_sc_hd_maj3_1)
  0.39     6.16    v _191_/X (sky130_fd_sc_hd_maj3_1)
  0.35     6.51    v _193_/X (sky130_fd_sc_hd_maj3_1)
  0.00     6.51    v _226_/D (sky130_fd_sc_hd_dfxt_1)
  6.51     6.51    data arrival time

  3.00     3.00    clock clk (rise edge)
  0.00     3.00    clock network delay (ideal)
  0.00     3.00    clock reconvergence pessimism
  3.00     3.00    ^ _226_/CLK (sky130_fd_sc_hd_dfxt_1)
 -0.13     2.87    library setup time
  2.87     2.87    data required time

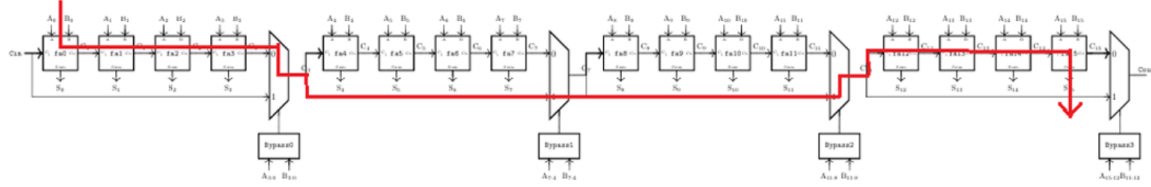
-----
  2.87     2.87    data required time
 -6.51    -6.51    data arrival time

-----
 -3.64    -3.64    slack (VIOLATED)
```

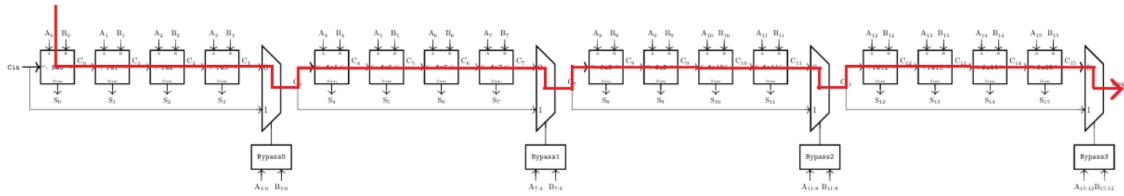
The obtained slack is : -3.64ns

Section 5: False path and exculsion of False path from STA:

The actual critical path will be the following:



The critical path obtained from STA report: (Highlighted in red)



The path reported above is a false path. The reason is, if the carry is going to be rippled as shown, it means the stages are in propagate mode. This implies block propagate is going to be high in all these stages. This will make the carry bypass the stages, leading to a different path and finally it will terminate through S15, which will actually be the critical path.

The following image shows specifying the above nodes as false path and running the analysis again:

Delay	Time	Description

0.00	0.00	clock clk (rise edge)
0.00	0.00	clock network delay (ideal)
0.00	0.00	^ _245_/CLK (sky130_fd_sc_hd__dfxtp_1)
0.29	0.29	v _245_/Q (sky130_fd_sc_hd__dfxtp_1)
0.37	0.67	v _167_/X (sky130_fd_sc_hd__maj3_1)
0.39	1.06	v _169_/X (sky130_fd_sc_hd__maj3_1)
0.39	1.45	v _171_/X (sky130_fd_sc_hd__maj3_1)
0.39	1.84	v _173_/X (sky130_fd_sc_hd__maj3_1)
0.39	2.23	v _175_/X (sky130_fd_sc_hd__maj3_1)
0.39	2.63	v _177_/X (sky130_fd_sc_hd__maj3_1)
0.39	3.02	v _179_/X (sky130_fd_sc_hd__maj3_1)
0.39	3.41	v _181_/X (sky130_fd_sc_hd__maj3_1)
0.13	3.54	v _204_/Y (sky130_fd_sc_hd__xnor2_1)
0.00	3.54	v _220_/D (sky130_fd_sc_hd__dfxtp_1)
	3.54	data arrival time
3.00	3.00	clock clk (rise edge)
0.00	3.00	clock network delay (ideal)
0.00	3.00	clock reconvergence pessimism
	3.00	^ _220_/CLK (sky130_fd_sc_hd__dfxtp_1)
-0.12	2.88	library setup time
	2.88	data required time

	2.88	data required time
	-3.54	data arrival time

	-0.66	slack (VIOLATED)

The actual slack along the true critical path is -0.66 ns, which is lower than the slack values found on the false paths. This is important because it demonstrates how false paths can give misleading results to both designers and timing analysis tools, potentially making the design seem slower than it actually is. Essentially, if false paths aren't properly excluded, they can cause designers to underestimate the highest possible operating frequency for a circuit. Since the validity of false paths depends on how the design behaves during runtime and specific functional conditions, static timing analysis tools are unable to detect them automatically.